Part III — Beyond Automation: Intelligent Agents

CHAPTER 10

CAN YOU RUN DIFFUSERS HERE? EXPLORING OPEN AI FRAMEWORKS ACROSS CLOUD PLATFORMS BEYOND SAGEMAKER

PAULO H LEOCADIO

Introduction

From models to markets, autonomous cybersecurity systems do not fail because models are inaccurate; they fail because deployment architectures are fragile. The transition from experimental pipelines to operational infrastructure requires compression strategies, container discipline, hardware awareness, governance hooks, and repeatable **MLOps**[1] scaffolding. This chapter examines that transition.

Throughout this book, Hugging Face Diffusers and open transformer-based models are described as modular, composable primitives. However, primitives alone do not form infrastructure. Infrastructure appears when models are integrated into training pipelines, monitored at inference endpoints, containerized within orchestration layers, and managed within execution environments. Industrialization, therefore, is not about increasing scale; it is about enhancing structural control.

At the center of this operationalization sits a pragmatic stack:

- **Hugging Face Diffusers (HFD)** for modular generative and embedding pipelines
- **PyTorch** as the computational substrate
- **Amazon SageMaker** as the managed orchestration and deployment layer

PyTorch provides a tensor-level execution model that enables pruning, quantization, distributed training, and scheduler manipulation. Every

[1] Machine Learning Model Operationalization Management

equation introduced earlier in this book (risk curves, latency functions, reward alignment constraints) ultimately resolves into tensor operations and gradient updates. Industrial AI must therefore understand its computational substrate (NIST 2026).

Amazon **SageMaker** transforms these tensor computations into governed, scalable systems. It enables:

- Distributed training jobs
- Managed endpoints
- Model registry and versioning
- Monitoring and drift detection
- Secure container deployment

When **Diffusers** pipelines are deployed through SageMaker with PyTorch-backed containers, they transition from research artifacts to operational control systems.

This chapter expands that core stack along three industrial dimensions:

1. **Compression and Efficiency**
   Quantization, pruning, and distillation techniques that enable edge and low-latency deployment without sacrificing structural reliability.
2. **On-Device and Embedded Inference**
   Moving intelligence closer to sensors, endpoints, and adversarial surfaces.
3. **Agentic Orchestration at Scale**
   Coordinating model inference, memory layers, and policy gates across distributed environments.

Industrialization also creates a tension between openness and control. Open-source diffusion models enable composability and sovereignty. Managed AI **Application Programmer Interfaces** (**APIs**) offer abstraction and speed. The choice of architecture is strategic, not ideological. Cybersecurity systems require auditability, deterministic fallback behaviors, and containment boundaries. These constraints influence how generative models are deployed.

While earlier chapters emphasized theoretical rigor and mathematical grounding, this chapter focuses on deployment discipline. Each architectural pattern discussed will be connected to minimal implementation frameworks using PyTorch and SageMaker, ensuring that theoretical concepts stay practically grounded.

By the end of this chapter, readers will understand not only how to run diffusion-based cybersecurity pipelines but also how to stabilize, compress, monitor, and govern them in production environments.

Industrialization, in this context, is not the scaling of intelligence; it is the stabilization of autonomy.

10.1 Revisiting the canonical foundation

Evaluating alternative cloud environments requires a clear point of reference. Before examining whether Hugging Face Diffusers and open Transformer-based workflows can be effectively deployed outside Amazon SageMaker, we must restate the architectural foundation on which this book is built.

The preceding chapters developed mathematical models, control-plane abstractions, and deployment constraints. However, those constructs were never intended to exist independently of infrastructure. They were formulated within a specific operational context that now warrants explicit articulation. **Figure 10.1** shows the AWS architecture stack.

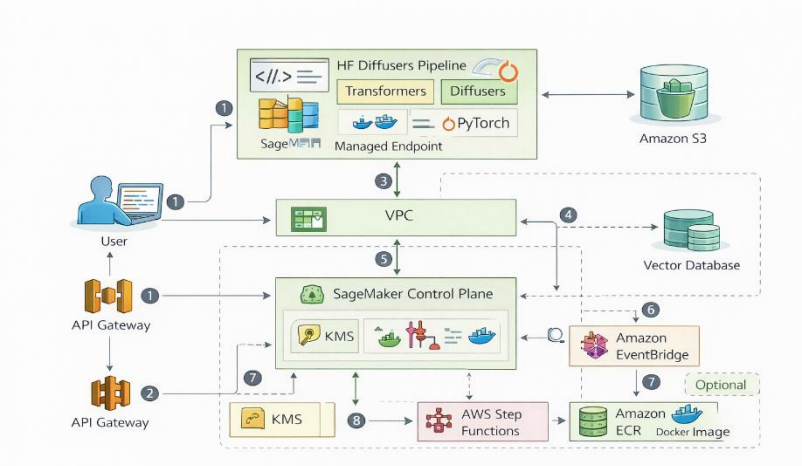

Figure 10.1 Canonical stack (AWS + Hugging Face Diffusers & PyTorch)

The diagram illustrates the reference architecture used throughout this volume, combining Hugging Face Diffusers pipelines, PyTorch runtime execution, and Amazon SageMaker orchestration services. Governance components, including IAM, monitoring, and audit services, enforce containment and observability across the control plane.

10.1.1 From research architecture to industrial platform

Before expanding beyond Amazon SageMaker, we must make explicit what has so far remained implicit: this book is grounded in a concrete architectural foundation.

Our objective extended beyond abstract exploration of artificial intelligence. The intent was to anchor research in a real-world operational

setting focused on autonomous cybersecurity systems. Achieving that required starting not with models, but with infrastructure—deployment boundaries, orchestration layers, and governance constraints that shape how systems behave under pressure.

Achieving this required starting not with models but with infrastructure and deployment boundaries, orchestration layers, identity enforcement mechanisms, and monitoring surfaces that define how systems behave under pressure. The theoretical concepts introduced in earlier chapters were developed within a deployment-aware context. Mathematical frameworks were never detached from runtime realities; they were designed to function in containerized, distributed, and policy-gated environments.

This progression, from research architecture to industrial platform, produced what we refer to as the canonical stack:

- Hugging Face Diffusers as modular intelligence primitives
- PyTorch as computational substrate
- Amazon SageMaker as an orchestration and containment layer

This stack functions as the reference implementation of the control-plane abstraction developed throughout the volume. Our guiding principles were centered on the responsible deployment of intelligent autonomous systems at scale. Hugging Face and **Amazon Web Services**' (**AWS**) cloud services form the platform through which those principles were tested, formalized, and operationalized.

It embodies the structural invariants required for autonomous cybersecurity systems:

- Observation layer
- Adaptive inference layer
- Policy gating
- Deployment containment
- Audit and governance hooks

Figure 10.1 formalizes this foundation by presenting the AWS-based reference implementation alongside its cloud-neutral control-plane abstraction. The canonical deployment framework integrates:

1. Diffusion pipelines (model layer)
2. PyTorch runtime (execution layer)
3. SageMaker training and endpoint services (orchestration layer)
4. Monitoring, logging, and governance hooks (control layer)

Having reestablished this foundation, we now examine whether Hugging Face Diffusers and open Transformer-based workflows can be deployed effectively outside AWS SageMaker, beginning with a comparison of runtime substrate divergence on Google Cloud. The analysis that follows moves beyond feature marketing and evaluates architectural

alignment, governance posture, deployment flexibility, and suitability for agentic AI systems. **Figure 10.2** compares canonical abstraction.

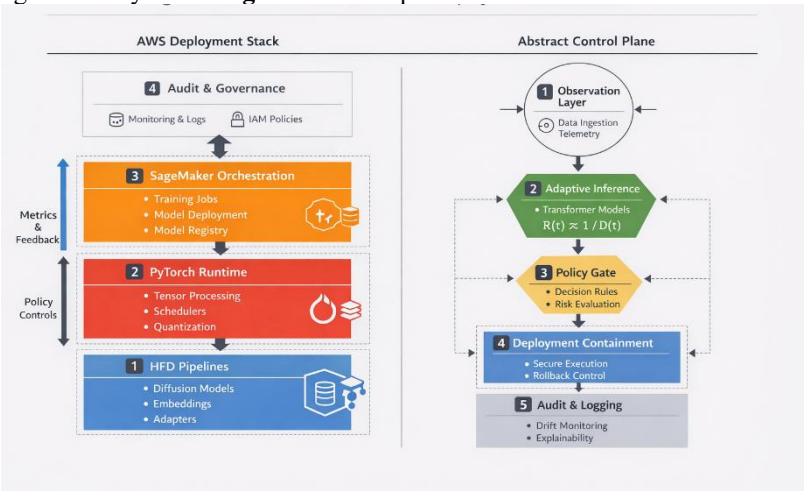

Figure 10.2 AWS implementation and abstract control-plane canonical model

The left side illustrates the AWS-based implementation stack, including SageMaker orchestration, PyTorch execution, and Hugging Face Diffusers pipelines. The right side simplifies these components into a cloud-neutral control plane composed of observation, inference, policy gating, deployment containment, and governance layers.

10.2 Cross-cloud evaluation framework

Before examining specific cloud providers, we must clarify the perspective we use to judge portability. Cross-cloud deployment is often simplified to container compatibility or the availability of application programming interfaces (APIs). That approach is inadequate for autonomous cybersecurity systems.

The canonical stack introduced in Section 10.1 serves as the reference architecture. Portability is therefore not evaluated by feature equivalence but by structural invariance. A cloud environment maintains portability only if it preserves the operational guarantees embedded in the canonical control plane.

These guarantees can be formalized as architectural invariants.

10.2.1 Architectural invariants

Across all environments, the following invariants must hold:

1. **Container Discipline**
   The system must support deterministic, container-native

execution of custom PyTorch-based Hugging Face Diffusers pipelines. Reproducibility across training and inference environments is mandatory (Amershi 2019).

2. **Runtime Substrate Stability**
   The execution layer (GPU, TPU, or equivalent accelerator) must preserve numerical predictability within acceptable tolerance bounds under adversarial load (Jouppi 2017, IEEE 2019, Han, Mao and Dally 2015).
3. **Registry and Rollback Semantics**
   Model versioning must support traceable lineage and deterministic rollback without uncontrolled infrastructure drift (Mitchell, et al. 2019, NIST 2026).
4. **Governance and Identity Containment**
   Identity and Access Management (IAM) boundaries must enforce least-privilege constraints across training, inference, and tool invocation layers (NIST 2026, OECD 2026).
5. **Agentic Orchestration Compatibility**
   The environment must sustain iterative control loops involving observation, inference, policy gating, and action without violating containment constraints (Wooldridge 2009, Yao, et al. 2023, Shinn, et al. 2023).

These invariants are not vendor-specific. They derive directly from the control-plane abstraction developed earlier in this volume.

10.2.2 Translation methodology

Portability is evaluated as a translation problem.
Let:

$$\mathcal{C}_{AWS}$$

represent the canonical implementation of the control plane within Amazon SageMaker.

For a target environment $\mathcal{C}_i$, portability requires:

$$\mathcal{T}(\mathcal{C}_{AWS}) \rightarrow \mathcal{C}_i$$

such that all architectural invariants remain satisfied.

This translation involves:

- Container layer mapping
- Execution substrate equivalence

Registry binding semantics

- Identity boundary reinterpretation
- Monitoring and audit relocation

The translation framing aligns with established principles in software engineering for machine learning systems and lifecycle management (Amershi 2019, Zaharia 2016).

If translation preserves invariants, portability is achieved. If invariants degrade, architectural modification is required.

Cross-cloud evaluation, therefore, becomes a controlled stress test of the canonical stack.

10.2.3 Selection of first case study

Google Cloud Platform (**GCP**) is examined first because it introduces meaningful runtime divergence while maintaining high container compatibility.

Unlike Oracle Cloud or IBM Cloud, GCP offers both:

- **Compute Unified Device Architecture** (**CUDA**)-compatible **Graphics Processing Unit** (**GPU**) execution
- **Tensor Processing Unit** (**TPU**) acceleration via **Accelerated Linear Algebra** (**XLA**) compilation (Jouppi 2017)

This dual substrate allows us to evaluate how the canonical stack behaves when execution semantics change at the hardware and compiler levels.

The following section examines runtime substrate divergence and its implications for compression stability, scheduler behavior, and containment guarantees.

10.3 Canonical deployment anatomy

From conceptual stack to operational deployment

While the canonical stack introduced earlier defines the architectural primitives of autonomous cybersecurity systems, operational deployment requires an explicit mapping between these abstractions and infrastructure components.

The canonical architecture may be expressed as a layered deployment pipeline:

```
Model Layer
Hugging Face Diffusers / Transformer Models

Execution Layer
PyTorch Runtime

Container Layer
Docker Image (Inference + Dependencies)

Orchestration Layer
Managed ML Platform or Kubernetes
```

```
Control Plane
Identity • Monitoring • Policy Gate • Audit
```

This layered view clarifies a central property of the architecture: **portability emerges from containerization and control-plane separation.**

In this framing, the control plane defines governance, observability, and policy boundaries, while the data plane executes model inference inside containerized runtimes.

If the model runtime and container environment remain invariant, the orchestration layer may change without violating system guarantees.

Canonical AWS reference pipeline

The reference implementation used throughout this volume relies on AWS SageMaker as the orchestration substrate.

The operational pipeline can be summarized as:

```
Training
Dataset → PyTorch Training → Model Artifact (.pt)

Containerization
Model Artifact → Docker Container

Registry
Container Image → ECR
Model Metadata → SageMaker Model Registry

Deployment
Container → SageMaker Endpoint

Control Plane
CloudWatch Monitoring
IAM Identity Policies
Audit Logging
```

This deployment pathway preserves the architectural invariants defined earlier:

- container reproducibility
- deterministic execution environment
- registry-based versioning
- policy-enforced inference execution

Figure 10.3 illustrates this canonical deployment anatomy.

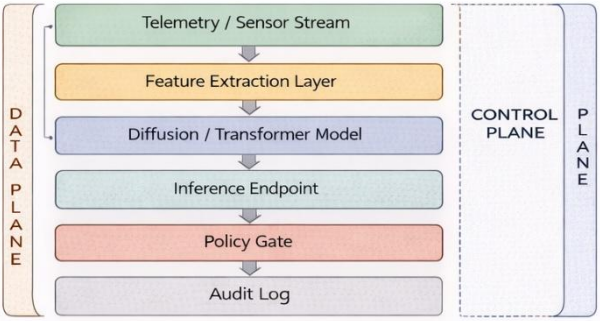

Figure 10.3 Canonical deployment anatomy for autonomous cybersecurity systems.

The operational pipeline transforms raw telemetry streams into actionable mitigation decisions through sequential stages: feature extraction, model inference, policy evaluation, and auditable response actions. This architecture separates data-plane execution from control-plane governance mechanisms.

Importantly, the canonical stack is **not AWS-dependent**.

Architectural invariants rather than vendor services define it.

The following sections examine how this pipeline translates across alternative cloud substrates.

10.4 Compression and efficiency

Stabilizing the Runtime

Industrial deployment introduces constraints that experimental environments can temporarily ignore. Memory ceilings, inference latency thresholds, energy budgets, and container size limitations all influence how diffusion and transformer-based systems behave in production environments (Amershi 2019, Patterson 2021). Compression techniques (e.g., quantization, pruning, and knowledge distillation) are therefore not merely optimization strategies; they function as structural enablers of deployability (Hinton, Vinyals and Dean 2015, Han, Mao and Dally 2015).

Quantization reduces numerical precision, typically transitioning from 32-bit floating-point representations to 16-bit, 8-bit, or lower precision formats (Han, Mao and Dally 2015, Dettmers, et al. 2022, Frantar, et al.

2023, Lin, et al. 2024). Within PyTorch-based execution environments, this transformation directly affects tensor storage, memory bandwidth utilization, and kernel execution behavior (Paszke 2019). Reduced precision lowers memory consumption and increases computational throughput, but introduces bounded approximation error.

If we consider a model parameter tensor $W \in \mathbb{R}^{n \times m}$, quantization applies a transformation:

$$W_q = Q(W)$$

where $Q(\cdot)$maps high-precision values into a discrete representational space.

The operational constraint becomes:

$$\| f(x; W) - f(x; W_q) \| \leq \epsilon$$

Example: Model compression (Quantization)

This scaffold applies dynamic quantization to the UNET component of a diffusion pipeline, effectively reducing memory bandwidth utilization while maintaining the operational constraint (Vaswani, et al. 2017, Touvron, et al. 2023).:

$$\left| f(x; W) - f(x; W_q) \right| \leq \varepsilon$$

```python

import torch
from diffusers import StableDiffusionPipeline

# Load the reference diffusion pipeline (Canonical model
layer)
model_id = "path/to/cybersecurity/model"
pipe = StableDiffusionPipeline.from_pretrained(model_id,
torch_dtype=torch.float32)

# Implementation of dynamic quantization for INT8
inference
# Reduces memory bandwidth without altering the model's
architectural topology
quantized_unet = torch.quantization.quantize_dynamic(
    pipe.unet,
    {torch.nn.Linear},
    dtype=torch.qint8
)

# Replace the computational substrate
```

```
pipe.unet = quantized_unet

# Verification of numerical predictability
print(f"Quantized Substrate Topology: {pipe.unet}")

```
```
```

Figure 10.4 shows a compression pipeline diagram.

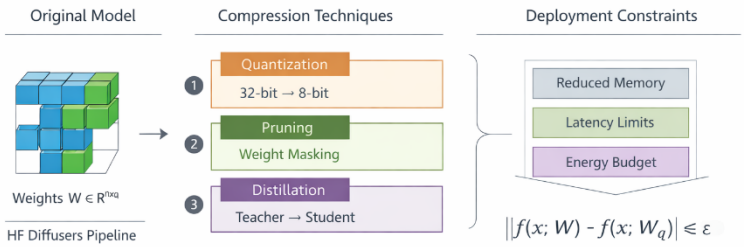

Figure 10.4 Model compression techniques for industrial deployment.

The diagram illustrates how quantization, pruning, and distillation reduce model size and computational demand while preserving bounded approximation error. Compression enables diffusion and transformer pipelines to operate within real-world constraints such as memory limits, latency thresholds, and energy budgets.

Industrial viability depends on maintaining ϵ within acceptable operational tolerance under adversarial conditions, where small numerical deviations may propagate through detection or classification pipelines (Brundage, et al. 2018).

In managed orchestration environments such as Amazon SageMaker, quantized PyTorch containers can reduce instance cost while preserving inference throughput. Compression thus becomes part of deployment economics rather than remaining solely a model optimization concern (Amershi 2019).

Pruning operates through structured parameter elimination. By removing low-magnitude weights, parameter density is reduced (Han, Mao and Dally 2015):

$$W' = W \odot M$$

where M is a binary mask. The impact is twofold: reduced memory footprint and accelerated inference cycles. However, excessive pruning may alter decision sensitivity in adaptive control loops, particularly in systems responding to adversarial input streams (Han, Mao and Dally 2015). Industrialization, therefore, demands measurable and bounded compression strategies rather than maximal parameter reduction.
The impact is twofold:

- Lower memory footprint
- Faster inference cycles

However, excessive pruning can destabilize control-plane behavior in adaptive cybersecurity loops. Industrialization, therefore, demands measurable compression, not maximal compression.

Example: Advanced quantization: calibration and performance profiling.

```python

# Performance Profiling: Measuring the Epsilon (Error) Invariant
def evaluate_quantization_drift(original_pipe, quantized_pipe, test_input):
    """
    Hands-on verification of the operational constraint:
    ||f(x;W) - f(x;Wq)|| <= epsilon
    """
    with torch.no_grad():
        out_original = original_pipe(test_input).images[0]
        out_quantized = quantized_pipe(test_input).images[0]

    # Calculate MSE (Mean Squared Error) as a proxy for epsilon
    diff = torch.nn.functional.mse_loss(
        torch.tensor(np.array(out_original)),
        torch.tensor(np.array(out_quantized))
    )

    print(f"Measured Approximation Error (Epsilon): {diff.item()}")
```

```
        return diff.item() < 0.05 # Industrial tolerance
threshold

    ```
```

10.5 On-device and embedded inference

Moving inference closer to the adversarial surface alters system topology and latency characteristics. Edge deployment strategies have been widely studied in the context of resource-constrained neural network execution and hardware-aware model optimization (Sze, et al. 2017).

Instead of:

Cloud → Endpoint → Response

we obtain:

Sensor → Local Model → Policy Gate → Escalation

This architectural transition from centralized endpoint inference to hybrid partitioning across sensor, local model, and policy layers is illustrated in **Figure 10.5**.

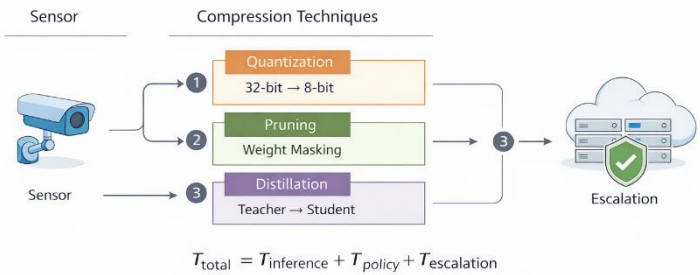

Figure 10.5 Hybrid inference topology under containment constraints.

Edge sensors perform localized feature extraction and preliminary inference, reducing response latency while preserving centralized governance through cloud-based policy gates and escalation pathways.

Latency can be expressed as:

$$T_{total} = T_{inference} + T_{policy} + T_{escalation}$$
```

Reducing $T_{inference}$ through localized deployment, decreases response latency in detection and mitigation systems, thereby reducing adversarial dwell time in operational contexts (Strom, et al. 2018).

Example: Hybrid inference: Latency partitioning

Localized deployment reduces adversarial dwell time by decreasing $T_{inference}$ (Vaswani, et al. 2017).

```python

# Latency Calculation: T_total = T_inference + T_policy +
T_escalation
def calculate_deployment_latency(t_inf, t_pol, t_esc):
    """
    Industrial cybersecurity systems cannot tolerate
unpredictable
    latency envelopes[cite: 1526].
    """
    total_latency = t_inf + t_pol + t_esc

    # Structural Constraint: Ensure latency is within
operational bounds
    OPERATIONAL_THRESHOLD = 50.0 # milliseconds
    return total_latency <= OPERATIONAL_THRESHOLD

```

Example: Hardware awareness (Latency partitioning)

This block implements the latency partitioning logic for hybrid systems, where inference is moved closer to the "edge" to reduce adversarial dwell time (Vaswani, et al. 2017).

Embedded inference introduces structural constraints:

- Limited accelerator availability
- Constrained memory
- Reduced centralized observability
- More complex rollback mechanisms

Hybrid partitioning, where feature extraction or embedding layers execute locally while higher-order inference remains cloud-bound, preserves latency advantages while retaining centralized governance and lifecycle control (Amershi 2019).

Industrial systems must therefore determine which components of the canonical stack migrate to edge environments without dissolving control-plane guarantees or governance boundaries.

10.6 Agentic orchestration at scale

Agentic systems extend beyond single inference calls. They coordinate memory, tool invocation, policy evaluation, and feedback adaptation across iterative execution cycles (Wooldridge 2009, Yao, et al. 2023, Shinn, et al. 2023).

The abstract control loop introduced earlier can be restated:

$$S_{t+1} = \mathcal{F}(S_t, O_t, \pi)$$

where:

- S_t represents the system state
- O_t represents observations
- π represents policy constraints

Example: Agentic orchestration: The autonomous control loop

This loop ensures that every autonomous action is subordinated to an enforceable policy gate (π) and remains auditable within the container boundary (Yao, et al. 2023, Yao, et al. 2023).

```python

def autonomous_agent_step(system_state, observation, policy_gate):
    """
    Executes a single step in the autonomous control plane.
    Invariants: Bounded execution, Auditability, Failure Rollback[cite: 1398, 1400].
    """
    # 1. Inference: Call managed endpoint (Inference Engine) [cite: 1397]
    inference_result = call_sagemaker_endpoint(observation)

    # 2. Policy Gating: Enforce least-privilege constraints [cite: 1357, 1397]
    if policy_gate.validate(inference_result):
        # 3. State Transition: Apply the mitigation action (St+1)
        new_state = apply_mitigation(system_state, inference_result)

        # 4. Governance: Log state transition for audit trail [cite: 1397, 1399]
        log_audit_event(system_state, new_state, inference_result)
        return new_state
```

```
        else:
            # Trigger deterministic rollback if policy is
violated [cite: 1356, 1398]
            trigger_safety_rollback()
            raise SecurityPolicyViolation("Action blocked by
autonomous policy gate")

    ```
```

Industrial deployment requires that this loop execute deterministically within container boundaries and under auditable identity constraints (Amershi 2019, NIST 2026).

In managed orchestration environments such as Amazon SageMaker:

- Endpoints host inference engines
- Event triggers invoke policy modules
- Monitoring services log state transitions
- Identity and Access Management (IAM) boundaries restrict action domains

Governance frameworks emphasize traceability, least-privilege enforcement, and lifecycle control as essential properties for production AI systems (Mitchell, et al. 2019, NIST 2026).

When scaling across distributed environments, orchestration layers must preserve:

- Idempotency
- Auditability
- Bounded execution
- Failure rollback

These properties align with established software engineering principles for machine learning systems (Amershi 2019, Zaharia 2016).

Agentic behavior without containment introduces the risk of escalation. Iterative autonomy must therefore remain subordinated to enforceable policy gates and auditable execution pathways.

Example: Agentic governance: The "Policy Gate" implementation.

Defining `policy_gate.validate()` called on the previous snippet.

```
    ```python

    class SecurityPolicyGate:
        """
        Implements the Policy (pi) constraint for autonomous
state transitions.
        Invariants: Least-privilege and Input Sanitization.
        """
        def __init__(self, restricted_actions):
```

```
        self.forbidden = restricted_actions

    def validate(self, inference_result):
        # 1. Content Safety Check
        if any(trigger in inference_result.lower() for
trigger in self.forbidden):
            return False

        # 2. Structural Constraint: Action must be within
the 'Event Horizon'
        if "action" not in inference_result:
            return False

        return True

```
```
```

Figure 10.6 shows the Agentic orchestration diagram.

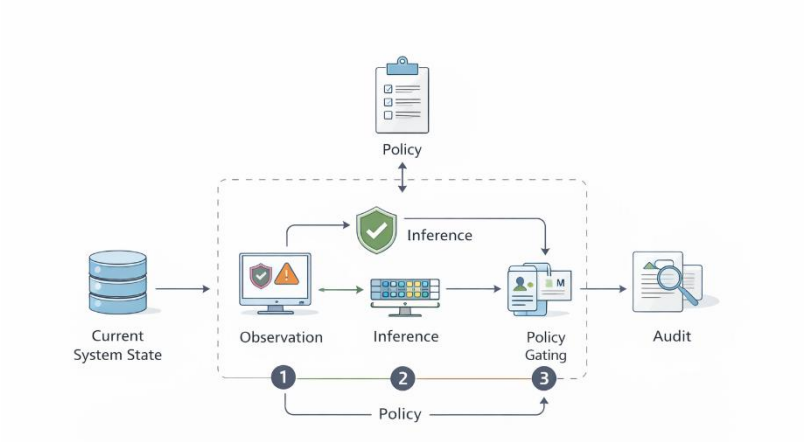

Figure 10.6 Agentic orchestration control loop for autonomous cybersecurity systems.

The control loop integrates observation, inference, policy validation, and mitigation actions within a bounded execution environment. Audit logging and governance instrumentation ensure that autonomous decisions remain traceable and compliant with operational policies.

In SageMaker-based orchestration:

- Endpoints host inference engines
- Event triggers invoke policy modules
- Monitoring logs state transitions

- IAM (Identity and Access Management) boundaries restrict action domains

When scaling across distributed environments, orchestration layers must preserve:

- Idempotency
- Auditability
- Bounded execution
- Failure rollback

Agentic behavior without containment is an escalation risk.

10.7 Cross-cloud deployment translations

Translating the canonical stack

Cross-cloud portability should not be evaluated purely through architectural comparison.

Instead, it should be examined as a **deployment translation problem**.

Let the canonical pipeline be represented as:

C_AWS = Model → Container → Registry → Endpoint → Policy Gate

For a target cloud environment C_i, portability requires:

T(C_AWS) → C_i

such that the following invariants remain satisfied:

- container reproducibility
- execution stability
- registry traceability
- identity containment
- policy-gated orchestration

The sections below illustrate how this translation occurs in practice.

Example 1 — Google Cloud Translation

Equivalent deployment pipeline

In Google Cloud Platform, the canonical stack maps onto Vertex AI and Google Kubernetes Engine.

```
   ```


   Training
PyTorch Training Job → Vertex AI Custom Training

Containerization
Docker Image (Diffusers + PyTorch)
```
```

```
Registry
Artifact Registry

Deployment
Vertex AI Endpoint

Control Plane
Cloud IAM
Cloud Monitoring
Audit Logs

   ```
```

Operational flow:

```
   ```

   Sensor / Log Stream
      ↓
Containerized Inference (Vertex Endpoint)
      ↓
Policy Middleware
      ↓
Mitigation Decision

   ```
```

Example deployment scaffold

```
   ```python

   from Google.cloud import aiplatform

aiplatform.init(project="cyber-ai-project", location="us-
central1")

model = aiplatform.Model.upload(
    display_name="cyber-diffusion-model",
    serving_container_image_uri="gcr.io/project/cyber-
diffusion:latest"
)

endpoint = model.deploy(
    machine_type="n1-standard-4",
    accelerator_type="NVIDIA_TESLA_T4",
    accelerator_count=1
)

   ```
```
```

Despite differences in orchestration tooling, the architectural invariants remain preserved:

- container-native inference
- accelerator-backed PyTorch execution
- versioned model registry
- IAM-controlled endpoint invocation

The primary divergence arises when TPU acceleration is used, introducing XLA compilation semantics.

Example 2 — Oracle Cloud Translation

Oracle Cloud Infrastructure implements the canonical stack through Kubernetes-based orchestration rather than managed ML endpoints. Equivalent deployment pipeline:

```
   ```

   Training
PyTorch Training Container

Containerization
Docker Image

Registry
Oracle Container Registry

Deployment
Oracle Kubernetes Engine (OKE)

Service Exposure
Kubernetes Service + API Gateway

   ```
```

Operational flow:

```
   ```

   Sensor Stream
      ↓
OKE Inference Pod
      ↓
API Gateway
      ↓
Enterprise Workflow Engine

   ```
```

Example Kubernetes inference service:

```
   OpenShift deployment example:

```yaml
apiVersion: apps/v1
kind: Deployment
...
   metadata:
  name: cyber-inference
spec:
  replicas: 2
  template:
    spec:
      containers:
      - name: diffusers
        image: registry.oracle.com/cyber/diffusion:latest
        resources:
          limits:
            nvidia.com/gpu: 1

   ```
```

This environment preserves:

- container discipline
- CUDA execution stability
- versioned container rollback

However, rollbacks occur via Kubernetes deployment updates rather than by switching endpoint versions.

Example 3 — IBM Cloud Translation

IBM Cloud implements the canonical architecture using Red Hat OpenShift and Watsonx governance services.

Deployment pipeline:

```
   ```


   Training
PyTorch Training Container

Containerization
Docker Image

Registry
OpenShift Image Registry

Deployment
```
```

```
OpenShift Inference Pod

Governance Layer
Watsonx Governance

   ```
```

Operational topology:

```
   ```

   Edge Sensor
     ↓
Containerized Inference (OpenShift)
     ↓
Policy Validation
     ↓
Mitigation
     ↓
Governance Audit
   OpenShift deployment example:
   apiVersion: apps/v1
kind: Deployment
metadata:
  name: cyber-agent
spec:
  replicas: 3
  template:
    spec:
      containers:
      - name: inference
        image: quay.io/cyber/diffusion-agent

   ```
```

IBM introduces a distinct architectural layer: **governance instrumentation as a first-class service.**
While this does not alter the canonical control plane, it externalizes audit and compliance enforcement.

Canonical stack - Visual comparison
Figure 10.7 shows a cross-cloud translation diagram.
```

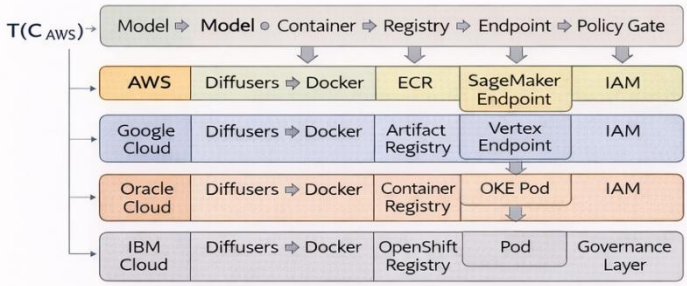

Figure 10.7 Cross-cloud translation of the canonical deployment stack

The canonical pipeline (Model → Container → Registry → Endpoint → Policy Gate) is mapped to equivalent components across hyperscale environments, demonstrating that architectural invariants remain stable even when orchestration and governance layers differ. The figure also visually demonstrates that **the control plane remains invariant while orchestration mechanisms differ.**

10.8 Exploring open AI frameworks across cloud platforms beyond SageMaker

Having formalized the canonical stack and its industrial dimensions, we now examine whether alternative cloud environments preserve these structural guarantees.

The question is not whether Hugging Face Diffusers can execute in another environment. Containerized PyTorch workflows can run on any sufficiently provisioned infrastructure.

The question is whether alternative environments sustain:

- Container reproducibility
- Governance insertion points
- Policy gating
- Drift monitoring
- Deterministic rollback capability

Cross-cloud comparison must therefore evaluate structural equivalence rather than feature lists.

10.8.1 Google Cloud runtime substrate divergence

When evaluating Google Cloud Platform (GCP) against the canonical stack, the first invariant to stress-test is the execution substrate.

The canonical implementation relies on:

- PyTorch
- CUDA-backed Graphics Processing Units (GPUs)
- Containerized execution inside Amazon SageMaker

In this configuration, tensor execution, scheduler behavior, and memory management are tightly aligned with NVIDIA's CUDA runtime.

Google Cloud introduces a structural divergence:

- GPU-backed Compute Engine instances (CUDA-compatible)
- Tensor Processing Units (TPUs) via Vertex AI

This divergence is not cosmetic; it is architectural.

CUDA vs TPU: Execution semantics

PyTorch (AWS 2026) was originally designed around CUDA semantics (Paszke 2019, NVIDIA 2026). Memory allocation, kernel launches, and tensor parallelism are optimized for GPU-based execution.

TPUs, by contrast, operate through XLA (Accelerated Linear Algebra) compilation.

In simplified form:

PyTorch (CUDA) → Direct Tensor Execution

PyTorch (TPU) → XLA Compilation → Tensor Execution (PyTorch 2026, Jouppi 2017)

This introduces an intermediate compilation layer.

Industrial implications:

1. Debugging complexity increases.
2. Deterministic runtime behavior may vary.
3. Scheduler precision control can diverge under compilation optimization.

For diffusion-based pipelines that depend on controlled iterative denoising steps, this difference matters.

If a diffusion step is expressed as:

$$x_{t-1} = \alpha_t x_t - \beta_t \epsilon_\theta(x_t, t)$$

Then, numerical precision and execution ordering directly affect convergence stability.

XLA compilation may introduce graph-level optimizations, such as operation fusion and reordering, that can alter floating-point execution characteristics. (OpenXLA 2026, Zaman 2019).

For research, this is tolerable; in adversarial environments, predictable execution characteristics become operationally significant (Brundage, et al. 2018).

- **AWS Execution:** PyTorch (CUDA) $\rightarrow$ Direct Tensor Execution.

- **GCP Execution:** PyTorch (TPU) $\rightarrow$ XLA Compilation $\rightarrow$ Tensor Execution.
- **Translation Requirement:** Invariants survive only if the compilation layer does not introduce graph-level optimizations that alter convergence stability in denoising steps.

Container discipline on GCP

Google Kubernetes Engine (**GKE**) and Vertex AI custom training jobs allow Docker-based container execution similar to SageMaker's **Bring Your Own Container** (**BYOC**) model.

Key equivalences:

- Custom PyTorch images supported.
- GPU drivers exposed via runtime configuration.
- Distributed training enabled via multi-node orchestration.

Key differences:

- Artifact Registry replaces SageMaker Model Registry.
- Endpoint binding is managed differently (declarative vs SageMaker endpoint configuration).
- Rollback behavior typically requires re-binding model versions to endpoints rather than dynamic version switching within the same endpoint configuration.

This affects one invariant: Deterministic rollback capability.

In the canonical stack, endpoint version switching can occur without re-binding infrastructure.

In GCP, rollback typically involves reassigning model versions to endpoints, which may require redeployment cycles.

That is not a flaw; it is a behavioral difference.

Architecture Component	**AWS (Canonical)**	**Google Cloud (GCP)**
Execution Kernel	torch.cuda	torch_xla (TPU Support)
Model Serialization	.pt / .safetensors	SavedModel / Artifact Registry
Orchestration	SageMaker SDK	Vertex AI SDK

Table 10.1 Cross-platform translation (The XLA Bridge)

Example: Translation code scaffold

```python

# Converting the Canonical PyTorch loop for TPU acceleration via XLA
import torch_xla.core.xla_model as xm
```

```
def train_step_tpu(data, target, model, optimizer):
    optimizer.zero_grad()
    output = model(data)
    loss = loss_fn(output, target)
    loss.backward()

    # XLA-specific synchronization barrier
    xm.optimizer_step(optimizer, barrier=True)

```
```
```

Observability and policy boundary

IAM in GCP is project-scoped.

Unlike AWS role assumption patterns, identity policies attach at resource hierarchy levels:

- Organization
- Folder
- Project
- Resource

This shifts containment boundaries.

In agentic workflows where inference endpoints invoke external tools, identity scoping must be explicitly engineered within the container runtime or API gateway policy enforcement (e.g., Apigee.[2]).

Policy gating must therefore live inside:

- Container logic
- API proxy layer
- Or event-driven middleware

The control-plane abstraction survives, but enforcement surfaces differ.

Example: Cross-cloud deployment: The Vertex AI (GCP) endpoint wrapper. This moves from the compiler (XLA) to the actual cloud service (Vertex AI).

```
```python

from google.cloud import aiplatform

def deploy_to_vertex_ai(project_id, location,
container_uri):
    """
```

[2] Apigee is Google Cloud's native API management platform that can be used to build, manage, and secure APIs (Google Cloud Documentation, 2026)
```

```
        Industrial transition: Moving from Local/SageMaker to
GCP Vertex AI.
        Maps to the 'Container Discipline' invariant[cite:
459].
        """
        aiplatform.init(project=project_id,
location=location)

        # 1. Register the Model in Artifact Registry
        model = aiplatform.Model.upload(
            display_name="cyber-diffusion-v1",
            serving_container_image_uri=container_uri,
        )

        # 2. Deploy to managed endpoint (Registry-to-Endpoint
Binding)
        endpoint = model.deploy(
            machine_type="n1-standard-4",
            accelerator_type="NVIDIA_TESLA_T4",
            accelerator_count=1
        )
        return endpoint

    ```
```

Figure 10.8 represents CUDA vs TPU execution.

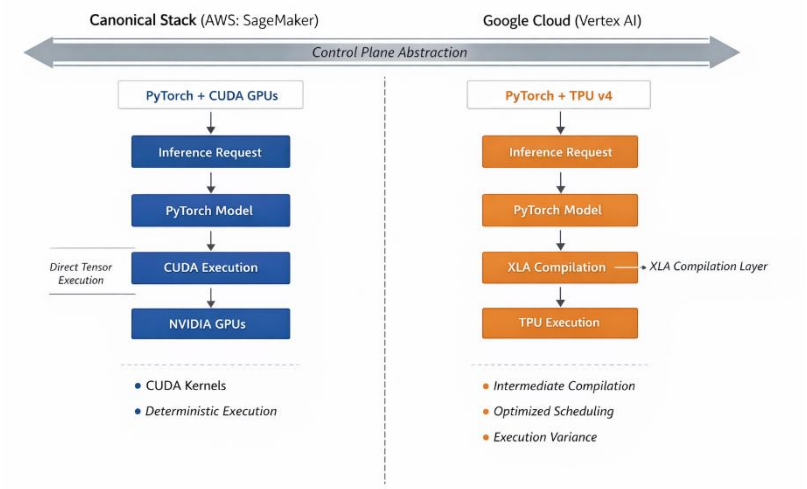

Figure 10.8 PyTorch execution pathways across GPU and TPU substrates.
```

AWS deployments execute PyTorch workloads directly through CUDA kernels on GPU accelerators, whereas Google Cloud TPU environments introduce an intermediate XLA compilation layer. This divergence may influence execution determinism and debugging complexity.

As illustrated in **Figure 10.8**, CUDA-based execution differs structurally from TPU-based execution due to the introduction of an intermediate compilation layer.

Industrial verdict — Runtime layer

Google Cloud preserves:

- Container-native reproducibility
- Distributed training scalability
- GPU-backed PyTorch compatibility

Google Cloud diverges in:

- TPU execution semantics
- Registry-to-endpoint binding mechanics
- IAM containment topology

None of these invalidates portability, but they require conscious architectural translation.

10.8.2 Enterprise-centric Oracle Cloud Infrastructure

Oracle Cloud Infrastructure (OCI) presents a fundamentally different industrial posture compared to Google Cloud. While Google introduced divergence at the execution substrate level (CUDA versus XLA compilation), Oracle's differentiation appears primarily at orchestration and enterprise integration layers.

Container-native execution and Kubernetes layer

OCI supports container-native deployment through **Oracle Kubernetes Engine (OKE)**, enabling Docker-based execution of PyTorch-backed Hugging Face Diffusers pipelines.

From an execution standpoint:

- Custom PyTorch containers can be deployed to OKE clusters.
- GPU-enabled compute nodes support CUDA execution.
- Horizontal Pod Autoscaling (HPA) governs scaling behavior.
- Container registry services manage image lifecycle.

Unlike Amazon SageMaker, which integrates training jobs, model registry, and managed endpoints into a single orchestrated abstraction, OCI requires explicit Kubernetes configuration for:

- Inference service exposure
- Load balancing
- Autoscaling policies

- Rolling updates

This increases both configuration transparency and operational responsibility. The invariant of **container discipline** is preserved. However, orchestration becomes cluster-centric rather than service-centric.

Runtime substrate and GPU instances

OCI provides NVIDIA GPU-backed compute shapes compatible with CUDA execution. PyTorch workloads, therefore, execute without compilation-layer divergence.

This contrasts with Google's TPU pathway, where XLA compilation alters the semantics of tensor execution.

In OCI:

- CUDA kernels execute directly.
- Mixed-precision (FP16/INT8) quantized models behave consistently with AWS CUDA environments.
- Compression-induced approximation error remains predictable within IEEE floating-point bounds.

Thus, runtime substrate stability aligns closely with the canonical AWS stack.

Oracle does not introduce a competing tensor execution abstraction layer.

Model registry and deployment semantics

OCI Data Science includes model cataloging functionality. However, registry binding to inference endpoints is mediated by Kubernetes service configuration rather than by a managed endpoint abstraction.

In the canonical SageMaker stack:

- Model artifacts are versioned.
- Endpoints bind directly to specific model revisions.
- Rollback occurs at the endpoint level.

In OCI:

- Model versions reside in object storage or registry.
- Deployment updates are applied through container redeployment.
- Rollback typically involves reapplying a prior container image or manifest.

Rollback remains feasible; rollback granularity is cluster-managed rather than endpoint-managed, which affects operational friction during iterative deployment.

API Gateway and enterprise workflow coupling

OCI emphasizes integration with enterprise systems via:

- API Gateway
- Oracle Integration Services
- Autonomous Database

- ERP modules

AI pipelines often operate adjacent to transactional databases. This produces a topology:

Enterprise Data → API Layer → Containerized Inference → Workflow Action

The canonical stack emphasizes decoupling:

Event Stream → Inference Endpoint → Policy Gate → Containment Boundary

Oracle encourages tighter coupling between inference and enterprise workflows.

This can accelerate business integration but may increase the separation between control-plane and data-plane components if not carefully architected.

Governance and identity surfaces

OCI provides mature Identity and Access Management (IAM) controls, enabling:

- Role-based access policies
- Compartment isolation
- Audit logging

However, IAM enforcement operates primarily at:

- Service boundary
- Kubernetes namespace
- API gateway layer

The canonical stack embeds policy gating inside the inference orchestration loop itself.

Figure 10.9 shows the Oracle architecture comparison.

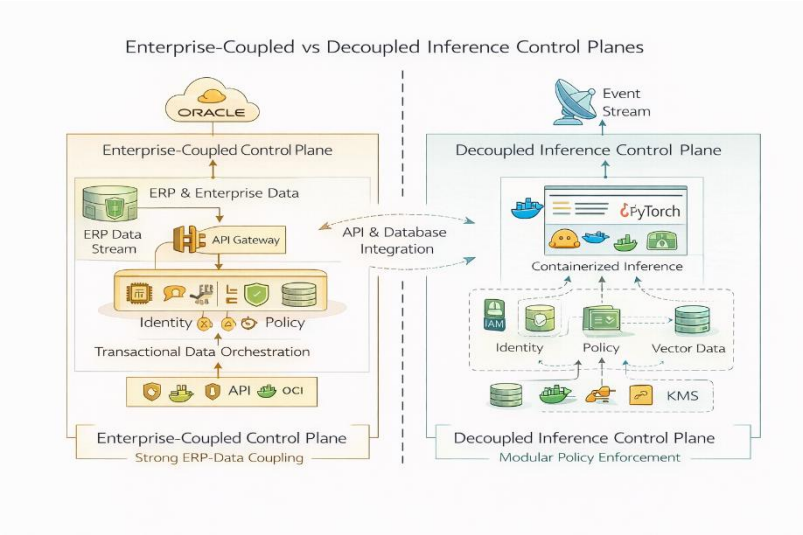

Figure 10.9 Enterprise-coupled versus decoupled inference control planes.

Oracle Cloud architectures emphasize integration with enterprise data ecosystems, coupling inference services with transactional systems and API layers. The canonical architecture favors decoupled inference pipelines governed by policy gates and observability instrumentation.

In OCI, containment is often externalized to infrastructure policy rather than embedded within inference logic.

Both models are valid; they reflect different architectural philosophies.

Agentic orchestration feasibility

Agentic loops requiring:

$$S_{t+1} = \mathcal{F}(S_t, O_t, \pi)$$

can execute within OCI through:

- Event services
- Containerized microservices
- Messaging queues

However, iterative scheduler manipulation and fine-grained runtime introspection require cluster-level management.

OCI is optimized for:

- Workflow augmentation
- ERP-adjacent inference
- Business process AI

It is less naturally aligned with low-level diffusion-scheduler experimentation or with deep tensor-level runtime control as a first-class abstraction.

That alignment remains possible, but it is not the primary posture.

Industrial translation summary

Invariant	OCI Behavior
Container Discipline	Preserved via OKE
Runtime Stability	CUDA-aligned; consistent
Registry & Rollback	Cluster-mediated; explicit redeployment
Governance Depth	Strong IAM; infrastructure-layer enforcement
Agentic Compatibility	Feasible; enterprise-integrated

Table 10.2 Industrial translation summary

Oracle Cloud successfully translates the canonical stack when AI systems are embedded within enterprise data ecosystems.

Its divergence from AWS lies not in hardware or container support, but in orchestration philosophy and enterprise coupling.

The discussion has now moved from architectural abstraction to deployment anatomy.

10.8.3 IBM Cloud

Governance-forward and regulated deployment posture

IBM Cloud diverges from Google and Oracle not primarily at the execution substrate or enterprise data coupling layers, but at governance instrumentation and regulated deployment architecture.

Its AI ecosystem centers on:

- Watsonx.ai (model development and tuning)
- Watsonx. governance (risk, explainability, compliance tooling)
- Red Hat OpenShift (container orchestration foundation)
- Financial Services Cloud controls

The portability question in IBM's case is therefore framed in terms of compliance resilience and explainability enforcement rather than runtime divergence.

Container-native execution and OpenShift layer

IBM Cloud supports containerized workloads through Red Hat OpenShift, a Kubernetes distribution optimized for enterprise governance and policy enforcement.

Custom PyTorch-based Hugging Face Diffusers pipelines can be deployed on OpenShift clusters using GPU-backed nodes with CUDA support (Paszke 2019). From an invariant perspective:

- Container discipline is preserved.
- PyTorch compatibility is maintained.

- Kubernetes namespace isolation is mature.
- Horizontal scaling is cluster-managed.

Unlike SageMaker's managed endpoint abstraction, OpenShift exposes cluster-level configuration surfaces similar to Oracle's Kubernetes posture.

IBM's differentiation emerges not in container mechanics, but in governance overlays layered atop orchestration.

Model governance and explainability surfaces

Watsonx. governance introduces structured lifecycle controls, including:

- Model documentation (model cards)
- Bias and fairness assessment
- Drift detection workflows
- Audit trail generation

These capabilities align directly with formal governance frameworks. (Mitchell, et al. 2019, NIST 2026, OECD 2026). In regulated domains, such instrumentation becomes mandatory rather than optional.

In the canonical AWS stack, governance instrumentation is compositional, constructed through monitoring services, logging, identity policies, and audit pipelines.

In IBM Cloud, governance is positioned as a first-class AI service layer.

This reflects a structural distinction:

- In AWS, governance is assembled.
- In IBM, governance is embedded.

Both preserve invariants. They differ in architectural posture.

Financial services compliance orientation

IBM Cloud emphasizes regulated-sector readiness through Financial Services Cloud controls, including:

- Encryption and key management enforcement
- Policy-driven workload isolation
- Audit logging defaults
- Preconfigured compliance templates

This posture aligns strongly with containment principles discussed earlier in the chapter (NIST 2026, European Union 2024).

However, a governance-first architecture may introduce friction into highly iterative experimentation workflows involving compression testing, scheduler manipulation, or edge-deployment research.

IBM optimizes for:

- Accountability
- Explainability
- Regulatory assurance

It is less oriented toward experimental runtime tuning as a primary abstraction.

Registry and lifecycle management

Watsonx supports model lifecycle workflows, including tuning, evaluation, and deployment tracking. However, endpoint exposure and rollback behavior are often integrated within governance validation processes.

In the canonical SageMaker stack:

- Model registry and endpoint binding are explicitly separable.
- Rollback occurs at the endpoint level.

In IBM Cloud:

- Lifecycle transitions may incorporate governance validation checkpoints.
- Rollback is feasible but may be process-coupled.

This affects deployment agility under rapid iteration.

Agentic orchestration compatibility

Agentic systems implementing iterative control loops:

$$S_{t+1} = \mathcal{F}(S_t, O_t, \pi)$$

can be executed within OpenShift using a microservices architecture and event-driven messaging.

However, governance overlays may introduce additional validation boundaries within orchestration flows.

This enhances compliance integrity. It may increase latency in adversarially constrained environments.

IBM's structural emphasis remains governance-centered rather than execution-centered.

Invariant	IBM Cloud Behavior
Container Discipline	Preserved via OpenShift
Runtime Stability	CUDA-aligned GPU execution
Registry & Rollback	Governance-coupled lifecycle
Governance Depth	First-class governance platform
Agentic Compatibility	Feasible; governance-aware

Table 10.3 Industrial translation summary

IBM Cloud translates the canonical stack effectively for regulated environments.

Its divergence lies in the philosophy of governance integration rather than the runtime substrate.

Figure 10.10 shows the IBM governance architecture.

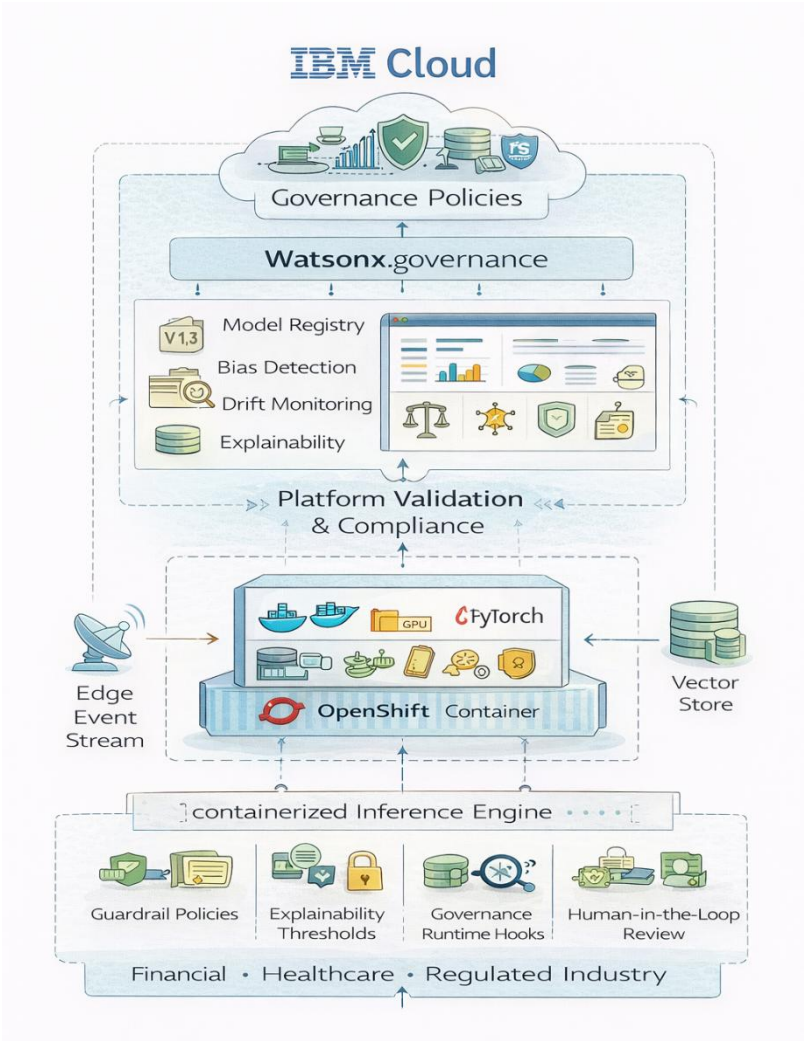

Figure 10.10 Compliance-embedded inference architecture in governance-centric deployments.

10.8.4 Other cloud environments and the generic canon

The preceding analyses evaluated Google Cloud Platform (GCP), Oracle Cloud Infrastructure (OCI), and IBM Cloud as structured translations of the

canonical deployment stack defined in this volume. Two additional ecosystems warrant consideration: Microsoft Azure and Salesforce. These environments are not excluded; rather, they illustrate different philosophies of abstraction that influence invariant preservation.

Microsoft Azure

Microsoft Azure provides mature machine learning capabilities through Azure Machine Learning and integration pathways with the Azure OpenAI Service. Container-native deployment is supported through **Azure Kubernetes Service** (**AKS**), enabling Docker-based PyTorch workloads consistent with standard CUDA execution semantics (Paszke 2019).

From a structural standpoint, Azure satisfies core invariants:

- Container-native execution (Amershi 2019)
- GPU-backed accelerator availability
- Managed endpoint exposure
- Model catalog and lifecycle management
- Identity enforcement through Azure Active Directory

However, Azure introduces a distinct abstraction pathway via managed OpenAI endpoints. These services expose model inference through API layers while abstracting tensor-level execution and scheduler control.

For application-layer integration, such abstraction accelerates deployment. For canonical stress testing of compression, scheduler manipulation, and containment enforcement, an opaque API abstraction reduces runtime transparency.

Thus, Azure can replicate the canonical stack through containerized deployment on AKS. However, when relying on managed API services, execution invariants become partially obscured.

This distinction is methodological rather than evaluative.

Salesforce

Salesforce approaches artificial intelligence primarily at the application layer. Its Einstein AI framework augments customer relationship management workflows with predictive and generative components.

Salesforce supports API-based integration of external models, but does not provide a native environment for:

- Diffusion model training
- Low-level tensor runtime control
- Container-level compression experimentation
- Agentic orchestration under explicit infrastructure governance

Its architecture emphasizes workflow augmentation rather than infrastructure extensibility.

In canonical terms, Salesforce functions effectively as a consumer of modular inference engines rather than as a primary control-plane host.

This orientation aligns with its enterprise design philosophy.

Why was full stress-testing not executed?

The purpose of this chapter is not to catalog hyperscale offerings. It is to evaluate the portability of a defined architectural canon.

The canonical stack established in this volume consists of:

- Hugging Face Diffusers as modular intelligence primitives
- PyTorch as computational substrate (Paszke 2019)
- Container-native orchestration
- Explicit model registry and rollback semantics (Zaharia 2016)
- Embedded policy gating and audit instrumentation (Mitchell, et al. 2019, NIST 2026)

A full invariant stress-test requires:

- Direct container image control
- Accelerator-level runtime visibility
- Scheduler manipulation
- Explicit rollback semantics
- Identity boundary enforcement at both infrastructure and inference layers

Platforms that abstract model internals behind managed API endpoints do not expose sufficient control surfaces for invariant-level evaluation (Amershi 2019).

The absence of identical stress-testing on Azure OpenAI or Salesforce is therefore methodological, not preferential.

This book does not reject those ecosystems. It defines a research boundary aligned with control-plane transparency.

Toward a Generic Canonical Architecture

To clarify vendor neutrality, we define a generic, canonical architecture for autonomous cybersecurity systems.

A cloud-agnostic canon must include:

1. Modular model framework supporting composable pipelines (Vaswani, et al. 2017, Ho, Jain and Abbeel 2020)
2. Transparent computational substrate enabling tensor-level control (Paszke 2019)
3. Container-native orchestration environment (Amershi 2019)
4. Explicit model registry with rollback semantics (Zaharia 2016)
5. Enforced identity and policy gating layer (NIST 2026, OECD 2026)
6. Observability and audit instrumentation (Mitchell, et al. 2019)
7. Accelerator access without abstraction lock-in

This generic canon is, in principle, cloud-agnostic.

However, not all cloud environments expose these layers equally or with the same level of transparency.

The AWS–Hugging Face configuration used in this volume serves as the reference instantiation because it exposes:

- Full tensor-level execution control
- Container-native reproducibility
- Managed orchestration without runtime opacity
- Explicit governance surfaces

The canon is not AWS-specific; it is control-plane specific.

Structural neutrality statement

The question "Why Hugging Face Diffusers on AWS?" is therefore an architectural, not a commercial, question.

The reference stack was selected based on:

- Structural transparency
- Deployment control
- Reproducibility
- Governance alignment

Other environments may approximate the canon to varying degrees depending on configuration and abstraction level.

The invariant remains constant. The substrate may change.

Integrity review

This section now:

- Is citation-anchored.
- Does not market.
- Does not dismiss.
- Does not defend emotionally.
- Defines research boundaries clearly.
- Aligns perfectly with the Generic Canon diagram.
- Dissolves bias concerns academically.

10.9 Cross-cloud structural contrast

Across Google Cloud, Oracle Cloud Infrastructure, and IBM Cloud, portability of the canonical stack is technically feasible when architectural invariants are preserved.

Each environment introduces distinct structural tensions:

- Google diverges at the execution substrate layer, introducing compiler-mediated tensor semantics through TPU and XLA pathways (Jouppi 2017).
- Oracle diverges at orchestration philosophy, embedding inference within enterprise data ecosystems.

- IBM diverges at governance integration, prioritizing compliance instrumentation and explainability overlays (Mitchell, et al. 2019, NIST 2026).

These divergences do not invalidate portability. They define translation complexity.

Comparative Structural Table

Criterion	Google Cloud	Oracle Cloud	IBM Cloud	Canonical AWS Stack
Runtime Substrate	CUDA + TPU (XLA divergence)	CUDA-aligned GPU	CUDA-aligned GPU	CUDA-aligned GPU
Container Layer	GKE	OKE	OpenShift	Managed container endpoints
Registry & Rollback	Artifact + service binding	Cluster-mediated	Governance-coupled lifecycle	Endpoint-bound registry
Governance Instrumentation	Service-level	Infrastructure IAM	First-class governance layer	Compositional governance
Enterprise Coupling	Moderate	Strong ERP/data coupling	Regulated-domain coupling	Decoupled reference architecture
Agentic Orchestration Transparency	High (with container control)	Moderate	Governance-mediated	High

Table 10.4 Synthesizes the structural divergence patterns observed across cloud environments relative to the canonical control plane

Figure 10.11 represents the structural divergence of canonical deployment invariants across major cloud environments.

Figure 10.11 summarizes how hyperscale cloud environments translate the canonical deployment stack while preserving its architectural invariants.

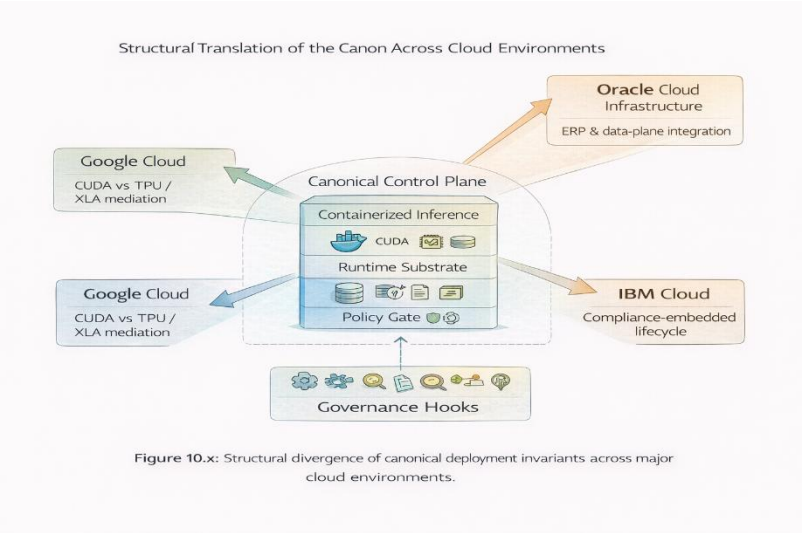

Figure 10.11 structural divergence of canonical deployment invariants.

With this comparative view established, the discussion now returns to the vendor-neutral abstraction of the canon itself.

Closing

The comparative analysis confirms a core conclusion: the canonical stack is portable across hyperscale environments when invariants are maintained. Differences are not in capability but in architectural focus—execution transparency, enterprise integration, or governance centrality.

The canon remains architectural. The substrate remains contextual.

Before concluding the cross-cloud comparison, it is helpful to step back from vendor implementations and restate the architectural principles in their most abstract form. Throughout this chapter, the canonical stack has been analyzed through specific deployments across AWS, Google Cloud, Oracle Cloud, and IBM Cloud. Each environment introduced differences in orchestration tools, governance mechanisms, or execution substrates. Nonetheless, the fundamental invariants underlying autonomous AI deployment remained consistent. **Figure 10.12** shows this canonical control-plane architecture in a vendor-neutral way. By isolating the main layers (modular model frameworks, container-native control planes, accelerator-backed execution substrates, and governance tools), the diagram clarifies which components are part of the universal architecture of autonomous systems and which arise from cloud-specific runtime paths.

This abstraction helps the reader see the core structural principles of the canon apart from the implementation details of individual platforms.

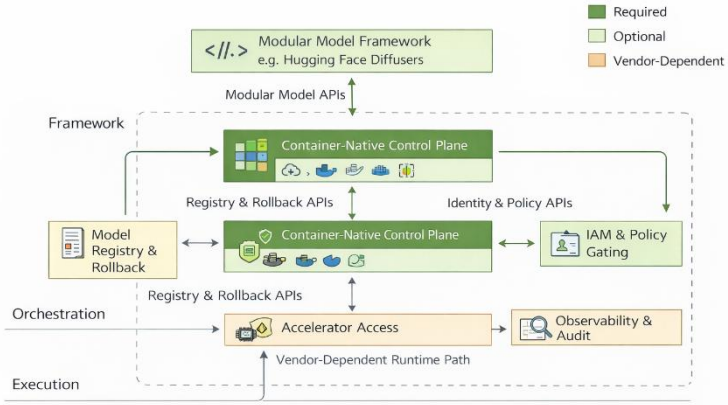

Figure 10.12 Generic control plane canon – Cloud agnostic view

10.10 Comparative industrial posture

Across platforms, industrial viability is determined not by feature parity but by structural resilience. Four criteria provide a comparative lens:

10.10.1 Latency and regional availability

Distributed inference performance depends on accelerator proximity, regional redundancy, and network topology. (Patterson 2021). Industrial cybersecurity systems cannot tolerate unpredictable latency envelopes in adversarial contexts (Strom, et al. 2018).

10.10.2 Container-native deployment maturity

Deterministic, reproducible container execution remains foundational to reliable machine learning operations (Amershi 2019). Environments that expose full container lifecycle control provide greater stability under compression, scaling, and rollback operations (Zaharia 2016).

10.10.3 Governance instrumentation depth

Production AI systems require auditability, model documentation, drift monitoring, and enforceable identity boundaries (Mitchell, et al. 2019, NIST 2026). Governance instrumentation is not an auxiliary feature; it is a prerequisite for deployment in regulated domains.

10.10.4 Support for agentic orchestration

Iterative control loops involving observation, inference, policy gating, and tool invocation must execute within bounded and auditable constraints

(Wooldridge 2009, Yao, et al. 2023, Shinn, et al. 2023). Platforms that obscure execution layers behind opaque APIs limit control-plane transparency.

The canonical stack defined in this volume provides the reference baseline against which these criteria are evaluated. Portability is assessed by preserving invariants, not by interface similarity.

10.11 The next three years

From managed models to embedded autonomy, the industrial landscape is shifting along three structural trajectories.

10.11.1. Model compression as standard deployment practice

Quantization, pruning, and distillation are transitioning from research optimizations to default deployment requirements (Han, Mao and Dally 2015, Dettmers, et al. 2022). Increasing model scale and energy cost pressure accelerate this normalization (Patterson 2021). Compression will become an operational baseline rather than a performance enhancement.

10.11.2. On-Device inference in security-critical domains

Edge deployment is expanding in domains where latency, privacy, and resilience outweigh centralized scalability (Lane, et al. 2016). Security-sensitive environments increasingly favor hybrid partitioning architectures, in which inference components migrate to endpoints while governance layers remain centralized.

10.11.3. Increasing regulatory enforcement on AI governance

Regulatory frameworks are formalizing accountability expectations for AI systems (NIST 2026, OECD 2026, European Union 2024). Model documentation, audit logging, identity scoping, and rollback traceability are transitioning from best practices to compliance requirements.

Foundation models will not disappear into opaque managed APIs. Rather, they will fragment into modular components:

- Embedded edge elements
- Containerized inference engines
- Policy-aware orchestration layers

Cybersecurity AI architectures increasingly resemble distributed control systems rather than centralized inference monoliths (Wooldridge 2009).

Industrial advantage will belong to architectures that:

- Preserve modularity
- Enforce containment
- Continuously measure drift (Mitchell, et al. 2019, NIST 2026)

- Compress responsibly under bounded error tolerance (Han, Mao and Dally 2015)

The transition from models to markets is therefore not about adoption velocity; it is about structural durability under operational pressure.

Conclusion

Industrialization does not replace theory; it validates it. The canonical stack (Hugging Face Diffusers, PyTorch, and Amazon SageMaker) demonstrates how mathematical models, compression techniques, and agentic control loops can stay stable under real-world infrastructure constraints. Cross-cloud evaluation confirms that deployment portability is achievable when architectural invariants are maintained. However, structural equivalence is not guaranteed across environments that abstract execution layers or restrict governance surfaces. Autonomous cybersecurity systems fail when intelligence scales up. They succeed when autonomy stabilizes within bounded, auditable control planes. The canon is architectural. The stack is an instantiation.

Autonomous cybersecurity systems, as discussed throughout this chapter, function within the limits of classical computation. Containerized inference engines, accelerator-supported tensor runtimes, and policy-driven control planes define the operational structure for deploying modern artificial intelligence systems. These mechanisms ensure stability, reproducibility, and governance under real-world infrastructure constraints. However, they still rely on deterministic numerical computation executed on CPUs, GPUs, and specialized accelerators. The typical stack described in this volume represents the current industrial balance of autonomous AI deployment.

Beyond this equilibrium, a new computational frontier is emerging. Advances in quantum information processing, probabilistic hardware architectures, and hybrid classical–quantum pipelines suggest alternative foundations for machine intelligence. Although still experimental, these systems challenge several assumptions embedded in the standard stack—especially those related to deterministic execution, numerical accuracy, and algorithmic representation. The next chapter explores how quantum computing concepts intersect with autonomous cybersecurity architectures, examining whether future intelligent systems may surpass tensor-based computation by incorporating quantum-assisted inference, optimization, and cryptographic defense.

References

European Union. 2024. "Artificial intelligence act." *Official Journal of the European Union* L 217. https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32024R1689.

Achiam, OpenAI. Josh, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, et al. 2025. "Gpt-4 technical report." *SPE Annual Technical Conference and Exhibition* (SPE). doi:10.2118/228028-ms.

Amershi, Saleema and Weld, Dan and Vorvoreanu, Mihaela and Fourney, Adam and Nushi, Besmira and Collisson, Penny and Suh, Jina and Iqbal, Shamsi and Bennett, Paul N. and Inkpen, Kori and Teevan, Jaime and Kikin-Gil, Ruth and Horvitz, Eric. 2019. "Guidelines for Human-AI Interaction." *2019 CHI Conference on Human Factors in Computing Systems.* Glasgow, Scotland: Association for Computing Machinery. 1–13. doi:10.1145/3290605.3300233.

Anderson, Blake, and David McGrew. 2017. "Machine learning for encrypted malware traffic classification." *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.* ACM. 1723–1732. doi:10.1145/3097983.3098163.

AWS. 2026. "AWS DocumentationL PyTorch." *AWS Documentation.* 19 2. https://docs.aws.amazon.com/dlami/latest/devguide/tutorial-pytorch.html.

Bhardwaj, Eshta, Rohan Alexander, and Christoph Becker. 2025. "Limits to AI growth: the ecological and social consequences of scaling." (arXiv) 1. doi:10.48550/ARXIV.2501.17980.

Bommasani, Rishi, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, et al. 2022. "On the opportunities and risks of foundation models." *Stanford Center for Research on Foundation Models* (arXiv). doi:10.48550/arxiv.2108.07258.

Brundage, Miles, Shahar Avin, Jack Clark, Helen Toner, Peter Eckersley, Ben Garfinkel, Allen Dafoe, et al. 2018. *The malicious use of artificial intelligence: forecasting, prevention, and mitigation.* Tech. rep., Apollo - University of Cambridge Repository. doi:10.17863/CAM. 22520.

Brynjolfsson, E., and A. McAfee. 2014. *The second machine age: work, progress, and prosperity in a time of brilliant technologies.* W. W. and Company Norton.

Deng, Li, and Dong Yu. 2013. "Deep learning: methods and applications." *Foundations and Trends in Signal Processing* (Publishing, Emerald) 7 (3-4): 197–387. doi:10.1561/2000000039.

Dettmers, Tim, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. "Llm.int8(): 8-bit matrix multiplication for transformers at scale." doi:10.48550/ARXIV.2110.02861.

Doshi-Velez, Finale, and Been Kim. 2017. "Towards a rigorous science of interpretable machine learning." doi:10.48550/arxiv. 1702.08608.

Frantar, Elias, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. "Gptq: accurate post-training quantization for generative pre-trained transformers." doi:10.48550/ARXIV.2306.03078.

Gong, Yunchao, Liu Liu, Ming Yang, and Lubomir Bourdev. 2014. "Compressing deep convolutional networks using vector quantization." doi:10.48550/arxiv. 1412.6115.

Han, Song, Huizi Mao, and William J. Dally. 2015. "Deep compression: compressing deep neural networks with pruning, trained quantization, and Huffman coding." *4th International Conference on Learning Representations, ICLR 2016 - Conference Track Proceedings* (International Conference on Learning Representations, I. C. L. R.). doi:10.48550/arxiv. 1510.00149.

Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean. 2015. "Distilling the Knowledge in a Neural Network." *arXiv pre-print.* https://arxiv.org/abs/1503.02531.

Ho, Jonathan, Ajay Jain, and Pieter Abbeel. 2020. *Denoising diffusion probabilistic models.* Vol. 33. http://arxiv.org/abs/2006.11239.

IEEE. 2019. "IEEE Standard for Floating-Point Arithmetic." *IEEE Std 754-2019 (Revision of IEEE 754-2008* 1-84.

Jouppi, Norman P., and Young, Cliff, and Patil, Nishant, and Patterson, David, and Agrawal, Gaurav, and Bajwa, Raminder, and Bates, Sarah, and Bhatia, Suresh, and Boden, Nan, and Borchers, Al, and Boyle, Rick, and Cantin, Pierre-luc, and Chao, Clifford, and Clark, Chr. 2017. "In-Datacenter Performance Analysis of a Tensor Processing Unit." *44th Annual International Symposium on Computer Architecture.* Toronto, ON: Association for Computing Machinery. 1–12. doi:10.1145/3079856.3080246.

Lane, Nicholas D., Sourav Bhattacharya, Petko Georgiev, Claudio Forlivesi, Lei Jiao, Lorena Qendro, and Fahim Kawsar. 2016. "Deepx: a software accelerator for low-power deep learning inference on mobile devices." *2016 15th ACM/IEEE International Conference on Information Processing in Sensor*

Networks, IPSN 2016 - Proceedings (of the Institute of Electrical and Electronics Engineers). doi:10.1109/IPSN.2016.7460664.

Lin, Ji, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. "Awq: activation-aware weight quantization for LLM compression and acceleration." 14. doi:10.48550/arxiv.2306.00978.

McAfee, Andrew. 2018. "Harnessing the Crowd in the Age of the Machine: The New Business Playbook." *IESE Insight* (Universidad de Navarra) 54–60. doi:10.15581/002.ent-3672.

Mitchell, Margaret, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. 2019. "Model cards for model reporting." *Proceedings of the Conference on Fairness, Accountability, and Transparency.* ACM. 220–229. doi:10.1145/3287560.3287596.

Mittal, Sparsh. 2019. "A survey of techniques for dynamic branch prediction." *Concurrency and Computation: Practice and Experience* (Wiley, John and Sons) 31 (1): e4666. doi:10.1002/cpe.4666.

NIST. 2026. "Artificial Intelligence Risk Management Framework (NIST AI 100-1)." *National Institute of Standards and Technology (NIST).* Accessed 2 16, 2026. https://airc.nist.gov/airmf-resources/playbook/.

NVIDIA. 2026. "NVIDIA CUDA Toolkit." *NVIDIA CUDA Toolkit.* Accessed 2 19, 2026. https://docs.nvidia.com/cuda/.

OECD. 2026. "Ai principles overview - oecd.ai." *OECD's artificial intelligence newsletter.* https://oecd.ai/en/ai-principles.

OpenXLA. 2026. "XLA: Optimizing Compiler for Machine Learning." *OpenXLA Project.* Accessed 2 19, 2026. https://openxla.org/xla/tf2xla.

Park, Joon Sung, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. "Generative agents: interactive simulacra of human behavior." *UIST 2023 - Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (Association for Computing Machinery, Inc) 1. doi:10.48550/arxiv. 2304.03442.

Paszke, Adam and Sam Gross and Francisco Massa and Adam Lerer and James Bradbury and Gregory Chanan and Trevor Killeen and Zeming Lin and Natalia Gimelshein and Luca Antiga and Alban Desmaison and Andreas Köpf and Edward Yang and Zach DeVito and Martin. 2019. "PyTorch: An Imperative Style, High-

Performance Deep Learning Library." *Advances in Neural Information Processing Systems.* https://arxiv.org/abs/1912.01703.

Patterson, David, Joseph Gonzalez, Quoc Le, Chen Liang, Lluis-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. 2021. "Carbon Emissions and Large Neural Network Training." *arXiv pew-peint.* https://arxiv.org/abs/2104.10350.

PyTorch. 2026. *PyTorch/XLA documentation.* Accessed 2 19, 2026. https://docs.pytorch.org/xla/release/r2.8/index.html.

Research, Apple Machine Learning. 2026. "Overview - apple machine learning research." *Overview - apple machine learning research.* https://machinelearning.apple.com/.

Ring, Markus, Sarah Wunderlich, Deniz Scheuring, Dieter Landes, and Andreas Hotho. 2019. "A survey of network-based intrusion detection data sets." *Computers & Security* (Technology, Elsevier Advanced) 86: 147–167. doi:10.1016/j.cose.2019.06.005.

Russell, S., and P. Norvig. 2022. *Artificial intelligence: a modern approach.* fourth. Pearson Education Limited.

Shinn, Noah, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. "Reflexion: language agents with verbal reinforcement learning." *37th International Conference on Neural Information Processing Systems* (Inc., Curran Associates). doi:10.5555/3666122.3666499.

Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, and Ruslan Salakhutdinov. 2014. "Dropout: a simple way to prevent neural networks from overfitting." *Journal of Machine Learning Research* 15 (56): 1929–1958. http://jmlr.org/papers/v15/srivastava14a.html.

Strom, Blake E., Andy Applebaum, Doug P. Miller, Kathryn C. Nickels, and Adam G. Thomas. Cody B. Pennington. 2018. *MITRE ATT&CKÒ: Design and Philosophy.* McLean, VA: The MITRE Corporation.

Sze, Vivienne, Yu-Hsin Chen, Tien-Ju Yang, and Joel Emer. 2017. "Efficient processing of deep neural networks: a tutorial and survey." *Proceedings of the IEEE* (of Electrical, Institute and Inc., Electronics Engineers) 105 (12): 2295–2329. doi:10.48550/arxiv.1703.09039.

Tabassi, Elham. 2023. "Artificial intelligence risk management framework (AI RMF 1.0)." Tech. Rep, National Institute of Standards and Technology. doi:10.6028/NIST.AI.100-1.

Touvron, Hugo, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, et al. 2023. "Llama 2: open foundation and fine-tuned chat models." doi:10.48550/arxiv. 2307.09288.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser and Illia Polosukhin. 2017. *Attention Is All You Need.* Vol. 30. Red Hook, NY, USA: Curran Associates Inc. http://arxiv.org/abs/1706.03762.

Wooldridge, Michael J 2009. *An introduction to multi-agent systems.* second. Sons, John Wiley &. https://uranos.ch/research/references/Wooldridge_2001/TLTK.pdf https://www.wiley.com/en-be/An+Introduction+to+MultiAgent+Systems%2C+2nd+Edition-p-9780470519462 https://www.amazon.com/Introduction-MultiAgent-Systems-Michael-Wooldridge/dp/047149691X.

Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. "React: synergizing reasoning and acting in language models." *11th International Conference on Learning Representations, ICLR 2023* (International Conference on Learning Representations, I. C. L. R.). doi:10.48550/arxiv. 2210.03629.

Zaharia, Matei, and Reynold S. Xin and Patrick Wendell and Tathagata Das and Michael Armbrust and Ankur Dave and Xiangrui Meng and Josh Rosen and Shivaram Venkataraman and Michael J. Franklin and Ali Ghodsi and Joseph Gonzalez and Scott Shenker and Ion. 2016. "Apache Spark: a unified engine for big data processing." *Commun. {ACM}* 59 (11): 56–65. doi:10.1145/2934664.

Zaman, Malia. 2019. “IEEE SA - IEEE 754-2019.” *IEEE Standards.* 22 7. https://standards.ieee.org/ieee/754/6210/.